

KSM

System Architecture Document

Enterprise Clean Architecture Specification for High-Volume .NET Platforms

This document defines a simple, explicit, and straightforward target architecture for KSM's .NET backend platform. It is written as a direct, buildable specification rather than an abstract discussion, so that engineers, database architects, security officers, and QA specialists can implement against it without ambiguity.

1. Purpose, Scope, and Architectural Objectives

1.1 Document Purpose

This document specifies the target technical architecture for KSM's modern, enterprise-grade backend services built on .NET. It translates functional, operational, and non-functional requirements into a concrete, buildable architecture, and serves as the primary technical baseline for software engineers, database architects, security officers, and quality assurance specialists.

1.2 Core Architectural Objectives

- ✚ **Maintainability & Decoupling:** Business logic is completely separated from user interface, database engines, and third-party infrastructure components.
- ✚ **Testability:** Use-case logic and domain rules can be validated via automated unit testing in complete isolation, without dependencies on physical databases or cloud resources.
- ✚ **Auditability & Financial Rigor:** All state-changing operations and financial movements are recorded immutably through append-only structures and high-speed audit logging.
- ✚ **Granular Security:** Fine-grained authorization is enforced systematically via module-action permissions rather than brittle, hardcoded role evaluations.
- ✚ **Extensibility:** Third-party services (payment gateways, communications, storage engines) are encapsulated behind stable adapter abstractions, allowing provider substitution without application-layer changes.

2. Architectural Style: 6-Tier Clean Architecture

The platform adopts a decoupled, six-layer Clean Architecture variant tailored for high-performance .NET applications. This design separates persistence management from external infrastructure integrations and centralizes common primitives into an independent layer.

The Inward Dependency Rule

All source-code dependencies point strictly inward toward the Domain core:

1. Outer layers (Presentation, Infrastructure, Persistence) depend on the inner layers (Application, Domain, Common).
2. The Application layer depends exclusively on the Domain and Common layers.
3. The Domain layer remains the sovereign center of the architecture and has no external dependencies.
4. No inner layer may ever reference a framework type, database provider, or protocol implementation originating from an outer layer.

3. Solution Partitioning and Assembly Reference Model

The standard KSM solution is partitioned into six distinct projects, ensuring strict isolation of concerns at compile time:

Project / Assembly	Layer Designation	Allowed Project References	Core Responsibilities
Core.Domain	Domain Layer	None (pure framework base class library only)	Enterprise entities, aggregates, enums, value objects, domain exceptions, and base entity abstractions.
Core.Common	Shared Kernel	Core.Domain	Application settings models, custom application exceptions, unified API response envelopes, authorization requirement definitions, and pagination utilities.
Core.Application	Application Layer	Core.Domain, Core.Common	Use-case orchestration, business service contracts and implementations, data transfer objects (DTOs), object mapping profiles, validation engines, and real-time hub contracts.
Core.Persistence	Persistence Layer	Core.Domain, Core.Common, Core.Application, Core.Infrastructure	Relational database context, entity configurations, database migrations, spatial data indexing, repository implementations, high-throughput micro-ORM audit repositories, and database-backed job runner configuration.
Core.Infrastructure	Infrastructure Layer	Core.Domain, Core.Common, Core.Application	External adapters: cloud object storage, SMS aggregators, payment gateways, transactional email engines, image processing utilities, and HTTP request managers.
Core.Api	Presentation / API Layer	Core.Common, Core.Persistence	ASP.NET Core hosting root, RESTful API controllers, real-time WebSocket endpoints, middleware pipeline composition, authentication schemes, API versioning, and Swagger generation.

Enforced Reference Governance

- ✚ **Core.Domain** must compile without any external NuGet packages or database libraries.
- ✚ **Core.Application** must never reference Entity Framework Core, SQL drivers, or web hosting libraries. Database interaction must occur strictly through repository or context interfaces declared within the Application layer itself.
- ✚ **Core.Infrastructure** must encapsulate third-party provider SDKs, exposing only technology-agnostic interfaces to the Application layer.
- ✚ **Core.Api** acts as the composition root, wiring dependencies via inversion of control, but individual controllers must communicate solely through application interfaces.

4. Layer Responsibilities & Component Architecture

4.1 Domain Layer (Core.Domain)

The Domain layer encapsulates all enterprise logic and state invariants:

- ✚ **Base Entity Contract:** Establishes identity through globally unique identifiers, paired with UTC creation and modification timestamps managed automatically by persistence interceptors.
- ✚ **Identity Aggregates:** Extends enterprise user identity abstractions with operational profiling, organizational memberships, and role assignments.
- ✚ **Relational Enums & Value Objects:** Enforces strongly typed representations of business concepts (statuses, categories, payment modes, approval tiers) to avoid primitive obsession.
- ✚ **Geospatial Value Models:** Integrates standardized geometry types (points, polygons) for geospatial tracking and spatial boundary definitions.

4.2 Application Layer (Core.Application)

The Application layer directs operational flow and coordinates use cases:

- ✚ **Direct Service Paradigm:** Employs cohesive, domain-focused application services representing specific business units (user management, order orchestration, wallet operations, customer service) behind explicit contracts.
- ✚ **Data Transfer Objects (DTOs):** Enforces distinct input models (requests) and output projections (responses), shielding the domain model from external exposure.
- ✚ **Validation Subsystem:** Implements automated validation rules covering data completeness, formatting, and structural constraints prior to use-case execution.
- ✚ **Object Transformation:** Utilizes automated object-mapping profiles to handle bidirectional conversion between Domain entities and DTOs.

4.3 Persistence Layer (Core.Persistence)

The Persistence layer provides physical storage abstractions:

- ✚ **Entity Framework Core Mapping:** Isolates database column names, foreign keys, cascade delete policies, and table naming conventions in external configuration files.
- ✚ **Schema Evolution:** Controls database structural changes entirely through reviewed, reproducible code migrations.
- ✚ **Spatial Database Capabilities:** Configures native database geospatial extensions to execute proximity queries and spatial computations directly inside the database engine.

4.4 Infrastructure Layer (Core.Infrastructure)

The infrastructure layer delivers external integration capabilities:

- ✚ **Cloud Storage Adapters:** Manages secure asset uploads, pre-signed download URLs, and metadata handling against cloud-hosted object stores.
- ✚ **Transactional Communications:** Implements reliable adapters for SMS verification codes, transactional notices, and template-driven email delivery.
- ✚ **Payment Engine Adapters:** Integrates third-party payment gateways for transaction initialization, checkout verification, and incoming webhook ingestion.

4.5 Common Shared Kernel (Core.Common)

Centralizes cross-cutting utility abstractions:

- ✚ **Uniform API Response Envelope:** Wraps all operational results in a standardized structure containing an HTTP status code, operational message, and projected payload.
- ✚ **Custom Application Exception Hierarchy:** Standardizes domain and operational error reporting, binding each exception type to specific HTTP status codes.
- ✚ **Configuration Options Objects:** Strongly binds configuration sections into structured models loaded at application startup.

4.6 Presentation Layer (Core.Api)

Exposes capabilities to web, mobile, and third-party consumers:

- ✚ **RESTful Controllers:** Thin controllers that validate model states, forward execution directly to application service interfaces, and return standard response wrappers.
- ✚ **Real-Time Communication Hubs:** Exposes bi-directional WebSocket endpoints for immediate notification distribution and live messaging.
- ✚ **API Versioning:** Enforces forward-compatible versioning through URL path segments, custom HTTP headers, and media type negotiation.

5. Data Architecture & Hybrid Access Strategy

5.1 Relational Schema Strategy

The database architecture employs a single, highly structured relational database supporting relational data models and geospatial coordinates. Logical separation is maintained via schema grouping or table categorization, grouping tables into identity, commerce, ledger, communication, and audit segments.

5.2 Hybrid Data Access Strategy: ORM + Micro-ORM

To balance developer productivity with high-performance operational throughput, the architecture uses a hybrid persistence model:

- ✚ **Entity Framework Core for Domain Writes and Complex Reads:** Manages aggregate roots, relational graphs, unit-of-work transactions, and business entity lifecycles with full change tracking.
- ✚ **Dapper (Micro-ORM) for High-Throughput Audit & Query Paths:** High-frequency operational audit records and large analytical read queries bypass change tracking completely. They execute via lightweight, parameter-mapped micro-ORM queries directly over database connections, minimizing memory allocation and CPU overhead.

5.3 Generic and Specialized Repositories

Data access is abstracted through a repository hierarchy:

- ✚ **Generic Repository:** Provides standard operations for aggregate retrieval, predicate-based filtering, eager loading of related entities, and pagination.
- ✚ **Spatial Repositories:** Encapsulates spatial calculations, allowing callers to locate entities within a specified radius or rank entities by proximity to a reference coordinate using native database indexing.
- ✚ **Specialized Repositories:** Encapsulates complex, optimized SQL queries and bulk operational routines for specific aggregates.

6. Financial Ledger & Immutable State Management

6.1 Double-Entry and Append-Only Accounting

To prevent financial discrepancies, balance drift, and race conditions, the system rejects mutable balance columns on financial entities.

6.2 Architectural Rules for Financial Operations

1. **Immutable Ledger Entries:** Every financial movement (deposit, fee deduction, payout, transfer) is recorded as an immutable ledger record.
2. **Dynamic Balance Derivation:** A wallet balance is always the computed sum of its cleared credit entries minus its cleared debit entries.
3. **Multi-State Lifecycle:** Entries transition through strict states (Pending, Cleared, Cancelled). Only cleared entries contribute to available balances.
4. **Traceable References:** Every entry maintains explicit foreign keys to the originating business transaction (order reference, payment identifier, payout authorization).
5. **Compensating Transactions:** Direct modification or deletion of posted entries is strictly prohibited. Errors are resolved solely by recording new compensating debit or credit entries.

7. Security & Authorization Architecture

7.1 Authentication Architecture

- ✚ **Stateless Token Management:** Authenticates requests using signed JSON Web Tokens (JWT) containing cryptographically verified identity claims and permission scopes.
- ✚ **Refresh Token Rotation:** Employs short-lived access tokens paired with secure, database-backed refresh tokens for continuous session renewal without credential exposure.
- ✚ **Multi-Transport Token Resolution:** Standard API requests extract tokens from the Authorization header. For WebSocket and SignalR handshakes where headers cannot be modified by standard browser clients, custom pipeline hooks extract and validate tokens from URL query strings.

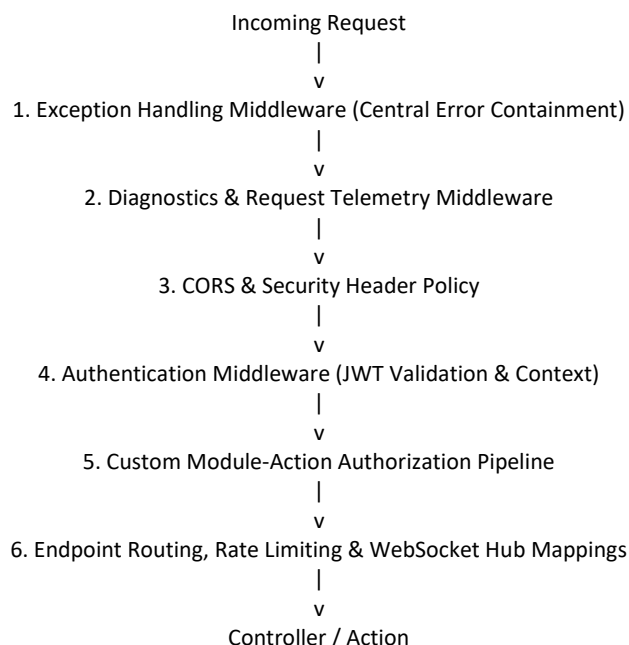
7.2 Fine-Grained Module-Action Authorization

Rather than relying on static, monolithic roles that require code changes whenever enterprise permissions evolve, the architecture enforces a granular module-action security matrix:

- ✚ **Permission Definition:** Permissions are defined as the combination of a functional Module and a specific Action (e.g., View, Create, Update, Delete, Approve).
- ✚ **Role-Permission Mapping:** Custom roles consist of dynamic collections of permissions stored in the database and assigned to user accounts.
- ✚ **Claim Bundling:** During authentication, the user's aggregate permissions are flattened into a structured claim list embedded in the token.
- ✚ **Declarative Permission Enforcement:** Endpoint controllers and actions are decorated with declarative permission attributes specifying the required module and action.
- ✚ **Central Authorization Handler:** A single authorization handler intercepts each request, validates the user's permission claims against the requirement, and rejects unauthorized actions before execution.

8. Cross-Cutting Concerns & Middleware Pipeline

The HTTP execution pipeline is assembled to ensure that security, telemetry, and error containment apply uniformly across all incoming requests.



8.1 Centralized Exception Handling

The exception middleware acts as a safety net:

- ✚ Intercepts all unhandled exceptions globally.
- ✚ Catches custom application exceptions, mapping them directly to the specified HTTP status code and user-friendly error message.
- ✚ Catches standard framework exceptions (bad requests, unauthorized access) and translates them into appropriate HTTP status codes.
- ✚ Catches unexpected internal exceptions, logs the full exception stack trace to secure server logs, and returns a sanitized generic error response to the client to prevent sensitive data leakage.

8.2 Logging and Observability

- ✚ **Structured Event Logging:** Employs structured logging sinks recording machine name, correlation IDs, timestamps, user identifiers, and client IP addresses.
- ✚ **Non-Blocking Audit Logging:** Crucial state-changing operations trigger audit trail entries dispatched directly through lightweight database commands, ensuring an unalterable activity history without impacting request throughput.

9. Real-Time Communications & WebSocket Lifecycle

9.1 Real-Time Topology

For immediate notification delivery, operational alerts, and live multi-user messaging, the architecture integrates full-duplex WebSocket hubs:

- ✚ **Notification Hub:** Dispatches real-time broadcast and user-targeted notifications across browser portals and mobile apps.
- ✚ **Collaboration & Messaging Hub:** Handles direct real-time communication between platform users, support representatives, and operational agents.

9.2 Real-Time Security Integration

- ✚ Handshake requests validate tokens passed via query parameters during connection establishment.
- ✚ Connected clients are organized into authenticated user channels and group-scoped rooms based on verified identity claims.
- ✚ Typed client interfaces ensure compile-time verification of event names and payloads transmitted across WebSockets.

10. Background Processing & Distributed Tasks

Long-running operations, scheduled maintenance routines, and financial reconciliations are decoupled from HTTP request threads:

- ✚ **Persistent Job Storage:** Background jobs, retry schedules, and recurring task metadata are stored in dedicated schema tables within the relational database, ensuring jobs survive server restarts.
- ✚ **Dedicated Execution Queues:** Jobs are segmented into prioritized queues (e.g., critical financial disbursements, standard communications, reporting exports).
- ✚ **Configurable Worker Concurrency:** Job runner servers allocate worker threads relative to processor capacity, ensuring background tasks do not starve HTTP request processing.
- ✚ **At-Least-Once Delivery & Retry Policies:** Failed jobs undergo exponential backoff retries with execution tracking visible through secure administration dashboards.

11. External Integration & Gateway Adapter Pattern

To guarantee that the core system remains decoupled from third-party APIs, all external dependencies are integrated via the Adapter Pattern:

- ✚ **Gateway-Agnostic Core Contracts:** The Application layer declares abstract interfaces defining the essential operations needed by the business (e.g., Initiate Payment, Verify Payment, Send SMS, Upload Document).
- ✚ **Isolated Infrastructure Adapters:** Dedicated classes inside the Infrastructure layer implement these contracts using third-party vendor SDKs or HTTP clients.
- ✚ **Provider Interchangeability:** Swapping or upgrading an external provider requires authoring a single new adapter class in Infrastructure and updating dependency injection configuration, without altering any domain or application logic.
- ✚ **Idempotent Webhook Processing:** Inbound webhooks from third parties are verified against cryptographic signatures and validated against local transaction tables to guarantee that duplicate webhook deliveries never cause duplicate processing.

12. Testing & Quality Assurance Strategy

The six-layer architecture establishes unambiguous testing boundaries:

Test Classification	Target Scope	Execution Environment	Validation Focus
Domain Unit Tests	Core Entities, Value Objects, Enums	In-memory; pure logic; zero mocks	Invariants, status transition rules, validation logic, and financial calculations.
Application Unit Tests	Application Services, Validators, Mapping Profiles	In-memory using mock/fake repositories and adapters	Use-case execution paths, validation failures, correct interaction with repository abstractions.
Persistence Integration Tests	Repositories, Database Context, Migrations	Disposable containerized database instances	EF Core mappings, spatial query translations, Dapper SQL queries, and migration integrity.
API Pipeline Integration Tests	Full Web API Pipeline, Middleware, Routing	In-memory test server with containerized database	Authentication verification, permission policy enforcement,

Test Classification	Target Scope	Execution Environment	Validation Focus
			status code mappings, and serialization rules.

13. Deployment, Infrastructure & Resiliency Architecture

13.1 Containerization & Hosting Strategy

- ✚ **Containerized Deployment:** The API is packaged as a lightweight Linux container running on Kestrel behind an edge reverse proxy (such as Nginx, Traefik, or cloud ingress controllers).
- ✚ **Stateless Application Tier:** The API processes are entirely stateless, delegating state to the database, distributed cache, and cloud storage. This enables rapid horizontal scaling behind load balancers.
- ✚ **Edge Proxy Responsibilities:** The edge proxy terminates TLS (TLS 1.2+ minimum), enforces DDoS protection, provides gzip/brotli compression, and handles request rate limiting.

13.2 Health Checks & Container Probes

The platform exposes standardized diagnostic endpoints:

- ✚ **Liveness Probe (/health/live):** Validates that the internal process is responsive; failure triggers container restart by the orchestrator.
- ✚ **Readiness Probe (/health/ready):** Validates database connectivity, storage engine availability, and cache responsiveness; failure removes the instance from the load-balancer pool until healthy.

14. Architecture Decision Records (ADR Log)

Decision Reference	Selected Approach	Architectural Justification
ADR-01: Six-Project Layering	Strict separation of Domain, Common, Application, Infrastructure, Persistence, and API.	Eliminates coupling between database mapping/migrations (Persistence) and external cloud/gateway SDKs (Infrastructure), preventing dependency pollution.
ADR-02: Direct Services Pattern	Interface-driven, scoped Application Services that directly implement each use case.	Reduces boilerplate code and class proliferation while maintaining strict testability, single-responsibility contracts, and dependency inversion — a simple, explicit alternative to indirect command/handler dispatch patterns.
ADR-03: Hybrid EF Core + Dapper Persistence	EF Core for complex relational workflows; Dapper for high-throughput audit trails and bulk reads.	Eliminates ORM tracking overhead on high-frequency operational paths while preserving full aggregate lifecycle management for core business logic.
ADR-04: Spatial Database Integration	Relational spatial engine integration via standard geometry abstractions.	Enables native, indexed distance and radius computations inside the database engine, avoiding the performance degradation of in-memory distance filtering.
ADR-05: Ledger-Based Accounting	Append-only ledger entries for all balance movements.	Guarantees mathematically auditable balances, eliminates update race conditions, and maintains a permanent audit trail of all financial actions.

Decision Reference	Selected Approach	Architectural Justification
ADR-06: Module-Action Permission Matrix	Declarative attribute-driven authorization based on dynamic module-action claims.	Allows dynamic role reconfiguration in database records without requiring application code changes or redeployments.
ADR-07: Persistent Background Job Queue	Relational database-backed Hangfire server with dedicated queue workers.	Guarantees reliable asynchronous execution and automatic retry handling within the primary database environment without introducing external queue dependencies.
ADR-08: Standardized Response Wrapper	Unified envelope with status code, message, and generic payload for all API responses.	Ensures consistent client-side consumption, uniform error handling, and predictable API integration across mobile, web, and external partners.